DAY 79 | DYNAMIC SEARCH APIS | JPA SPECIFICATIONS | SQL INJECTION & WILDCARD SAFE

# Dynamic Multi-Criteria Search APIs

Study Notes & Planned Implementation Blueprint: Awakening Existing AccountSearchRequest & CustomerSearchRequest DTOs, Building Type-Safe Spring Data JPA 4.1 Specifications, Safe LIKE Wildcard Escaping, and Unifying Validation + Pagination + Sorting.

| PROJECT | QUERY ENGINE | SPRING DATA | SECURITY |
|---|---|---|---|
| **Enterprise Digital Banking** | **JpaSpecificationExecutor** | **4.1.0 (unrestricted())** | **LIKE Wildcard Escaper** |

## 1. Core Concepts: Derived Queries vs. `@Query` vs. JPA `Specification`

Our codebase already contains two scaffolded search DTOs waiting to be wired: `AccountSearchRequest` (7 optional filters) and `CustomerSearchRequest` (7 optional filters), plus we need multi-filter transaction and audit log searches. Supporting 7 optional filters creates $2^7$ = 128 filter combinations!

| Search Approach | How It Works | Verdict for Enterprise Banking |
|---|---|---|
| **Derived Method Names** | `findByCityAndKycStatusAndFirstName ...` | AVOID FOR MULTI-FILTER Combinatorial explosion; acceptable only for simple 1-field lookup. |
| **Static JPQL `(:x IS NULL OR ...)`** | Huge hardcoded `@Query` checking null on every parameter. | SUBOPTIMAL Clutters SQL plans; MySQL optimizer often skips B-Tree indexes on `OR :x IS NULL` tricks. |
| **JPA `Specification<T>`** | Dynamically appends ONLY the non-null predicates via CriteriaBuilder, then compiles cleanly into parameterized JDBC SQL! | ENTERPRISE STANDARD Clean SQL containing only active filters, works seamlessly with Day 77 `Pageable` & Day 78 `SortValidator`! |

```
GET /api/v1/transactions/search?accountId=1&transactionType=TRANSFER&minAmount=500
    &page=0&size=20&sort=amount,desc
    │
    ├─→ Day 76 Bean Validation (@Valid @ModelAttribute TransactionSearchRequest)
    │     Checks: minAmount ≤ maxAmount, startDate ≤ endDate, @Positive IDs
    │
    ├─→ Day 78 SortValidator.validate(pageable, ALLOWED_TX_SORT_FIELDS, defaultSort)
    │
    ├─→ Day 79 TransactionSpecification.withFilters(request)
    │     Dynamically generates ONLY 3 SQL predicates (skips null remarks/reference/status):
    │     WHERE (source_account_id = ? OR destination_account_id = ?)
    │       AND transaction_type = ? AND amount ≥ ?
    │
    └─→ repository.findAll(spec, validatedPageable) ──→ PagedResponse<TransactionSummaryResponse>
```

## 2. Real Codebase Discovery: Already-Existing Search DTOs

Inspecting `src/main/java` uncovered two pre-existing search DTOs that have never been connected to repositories or controllers:

- `AccountSearchRequest.java`: `customerId`, `accountNumber`, `accountType`, `accountStatus`, `branch`, `minBalance`, `maxBalance`.
- `CustomerSearchRequest.java`: `customerId`, `firstName`, `lastName`, `phoneNumber`, `city`, `state`, `kycStatus`.

In Day 79, we harden both DTOs (cross-field range checks + `@Size` bounds), create `TransactionSearchRequest` & `AuditSearchRequest`, and wire all four end-to-end!

> **Security Subtlety — LIKE Wildcard Injection (ReDoS / Full Scan):** Criteria `cb.like(...)` prevents SQL injection via bind variables, but if a user sends `?branch=%` or `?firstName=_____%`, SQL treats `%` and `_` as wildcards and forces full table scans! We escape `\`, `%`, and `_` cleanly before wrapping in `%...%`.

## 3. Planned Code: Safe Helper, Repositories & Validated Search DTOs

src/main/java/ … /common/util/SpecificationHelper.java — NEW

```java
package org.example.enterprisedigitalbankingsystem.common.util;

import jakarta.persistence.criteria.CriteriaBuilder;
import jakarta.persistence.criteria.Expression;
import jakarta.persistence.criteria.Predicate;

public final class SpecificationHelper {

    private static final char ESCAPE_CHAR = '\\';

    private SpecificationHelper() {}

    /** Escapes SQL LIKE wildcards ('%', '_', '\') and performs case-insensitive contains match. */
    public static Predicate containsIgnoreCase(
            CriteriaBuilder cb, Expression<String> field, String rawInput) {
        String escaped = rawInput.trim().toLowerCase()
                .replace("\\", "\\\\")
                .replace("%", "\\%")
                .replace("_", "\\_");
        return cb.like(cb.lower(field), "%" + escaped + "%", ESCAPE_CHAR);
    }

    public static boolean hasText(String value) {
        return value ≠ null && !value.isBlank();
    }
}
```

Repositories: Enable JpaSpecificationExecutor<T> — MODIFIED

```java
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;

public interface AccountRepository extends JpaRepository<Account, Long>, JpaSpecificationExecutor<Account> {
 … }
public interface CustomerRepository extends JpaRepository<Customer, Long>, JpaSpecificationExecutor<Customer>
{ … }
public interface TransactionRepository extends JpaRepository<Transaction, Long>,
JpaSpecificationExecutor<Transaction> { … }
public interface AuditRepository extends JpaRepository<AuditLog, Long>, JpaSpecificationExecutor<AuditLog> {
 … }
```

AccountSearchRequest.java (Updated) & TransactionSearchRequest.java (New) — MODIFIED / NEW

```java
// AccountSearchRequest.java — adding Day 76 Bean Validation range & length rules
@Positive(message = "Customer id must be positive") private Long customerId;
@Size(max = 30) private String accountNumber;
private AccountType accountType;
private AccountStatus accountStatus;
@Size(max = 100) private String branch;
@DecimalMin("0.00") private BigDecimal minBalance;
@DecimalMin("0.00") private BigDecimal maxBalance;

@AssertTrue(message = "minBalance cannot exceed maxBalance")
public boolean isBalanceRangeValid() {
    if (minBalance == null || maxBalance == null) return true;
    return minBalance.compareTo(maxBalance) ≤ 0;
}

// TransactionSearchRequest.java — new multi-criteria transaction & date range filter DTO
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class TransactionSearchRequest {
```

```
    @Positive(message = "Account id must be positive") private Long accountId;
    @Size(max = 64) private String transactionReference;
    private TransactionType transactionType;
    private TransactionStatus transactionStatus;
    @DecimalMin("0.01") private BigDecimal minAmount;
    @DecimalMin("0.01") private BigDecimal maxAmount;
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE_TIME) private LocalDateTime fromDate;
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE_TIME) private LocalDateTime toDate;
    @Size(max = 100) private String remarksKeyword;

    @AssertTrue(message = "minAmount cannot exceed maxAmount")
    public boolean isAmountRangeValid() {
        return minAmount == null || maxAmount == null || minAmount.compareTo(maxAmount) ≤ 0;
    }

    @AssertTrue(message = "fromDate cannot be after toDate")
    public boolean isDateRangeValid() {
        return fromDate == null || toDate == null || !fromDate.isAfter(toDate);
    }
}
```

## 4. Planned Code: Spring Data 4.1 `Specification` Builders

Notice we use a clean `List<Predicate>` inside the lambda (avoiding deprecated `Specification.where(null)` from older Spring Data 2/3 codebases).

src/main/java/ ... /transaction/specification/TransactionSpecification.java NEW

```
package org.example.enterprisedigitalbankingsystem.transaction.specification;

import jakarta.persistence.criteria.Predicate;
import org.example.enterprisedigitalbankingsystem.common.util.SpecificationHelper;
import org.example.enterprisedigitalbankingsystem.transaction.dto.request.TransactionSearchRequest;
import org.example.enterprisedigitalbankingsystem.transaction.entity.Transaction;
import org.springframework.data.jpa.domain.Specification;
import java.util.ArrayList;
import java.util.List;

public final class TransactionSpecification {
    private TransactionSpecification() {}

    public static Specification<Transaction> withFilters(TransactionSearchRequest filter) {
        return (root, query, cb) → {
            List<Predicate> predicates = new ArrayList<>();
            if (filter == null) return cb.conjunction();

            if (filter.getAccountId() ≠ null) {
                Predicate isSource = cb.equal(root.get("sourceAccount").get("id"), filter.getAccountId());
                Predicate isDest = cb.equal(root.get("destinationAccount").get("id"), filter.getAccountId());
                predicates.add(cb.or(isSource, isDest));
            }
            if (SpecificationHelper.hasText(filter.getTransactionReference())) {
                predicates.add(cb.equal(root.get("transactionReference"),
                        filter.getTransactionReference().trim()));
            }
            if (filter.getTransactionType() ≠ null) {
                predicates.add(cb.equal(root.get("transactionType"), filter.getTransactionType()));
            }
            if (filter.getTransactionStatus() ≠ null) {
                predicates.add(cb.equal(root.get("transactionStatus"), filter.getTransactionStatus()));
            }
            if (filter.getMinAmount() ≠ null) {
                predicates.add(cb.greaterThanOrEqualTo(root.get("amount"), filter.getMinAmount()));
            }
            if (filter.getMaxAmount() ≠ null) {
                predicates.add(cb.lessThanOrEqualTo(root.get("amount"), filter.getMaxAmount()));
```

```
            }
            if (filter.getFromDate() != null) {
                predicates.add(cb.greaterThanOrEqualTo(root.get("createdAt"), filter.getFromDate()));
            }
            if (filter.getToDate() != null) {
                predicates.add(cb.lessThanOrEqualTo(root.get("createdAt"), filter.getToDate()));
            }
            if (SpecificationHelper.hasText(filter.getRemarksKeyword())) {
                predicates.add(SpecificationHelper.containsIgnoreCase(
                        cb, root.get("remarks"), filter.getRemarksKeyword()));
            }
            return cb.and(predicates.toArray(new Predicate[0]));
        };
    }
}
```

AccountSpecification.java & CustomerSpecification.java — NEW

```
// AccountSpecification.java (wiring AccountSearchRequest)
public static Specification<Account> withFilters(AccountSearchRequest f) {
    return (root, query, cb) -> {
        List<Predicate> p = new ArrayList<>();
        if (f == null) return cb.conjunction();
        if (f.getCustomerId() != null)     p.add(cb.equal(root.get("customer").get("id"),
f.getCustomerId()));
        if (hasText(f.getAccountNumber())) p.add(cb.equal(root.get("accountNumber"),
f.getAccountNumber().trim()));
        if (f.getAccountType() != null)    p.add(cb.equal(root.get("accountType"), f.getAccountType()));
        if (f.getAccountStatus() != null)  p.add(cb.equal(root.get("accountStatus"), f.getAccountStatus()));
        if (hasText(f.getBranch()))        p.add(containsIgnoreCase(cb, root.get("branch"), f.getBranch()));
        if (f.getMinBalance() != null)     p.add(cb.greaterThanOrEqualTo(root.get("balance"),
f.getMinBalance()));
        if (f.getMaxBalance() != null)     p.add(cb.lessThanOrEqualTo(root.get("balance"),
f.getMaxBalance()));
        return cb.and(p.toArray(new Predicate[0]));
    };
}

// CustomerSpecification.java (wiring CustomerSearchRequest + optional keyword convenience)
public static Specification<Customer> withFilters(CustomerSearchRequest f) {
    return (root, query, cb) -> {
        List<Predicate> p = new ArrayList<>();
        if (f == null) return cb.conjunction();
        if (f.getCustomerId() != null)   p.add(cb.equal(root.get("id"), f.getCustomerId()));
        if (hasText(f.getFirstName()))   p.add(containsIgnoreCase(cb, root.get("firstName"),
f.getFirstName()));
        if (hasText(f.getLastName()))    p.add(containsIgnoreCase(cb, root.get("lastName"),
f.getLastName()));
        if (hasText(f.getPhoneNumber())) p.add(cb.equal(root.get("phoneNumber"), f.getPhoneNumber().trim()));
        if (hasText(f.getCity()))        p.add(containsIgnoreCase(cb, root.get("city"), f.getCity()));
        if (hasText(f.getState()))       p.add(containsIgnoreCase(cb, root.get("state"), f.getState()));
        if (f.getKycStatus() != null)    p.add(cb.equal(root.get("kycStatus"), f.getKycStatus()));
        return cb.and(p.toArray(new Predicate[0]));
    };
}
```

## 5. Planned Code: Unified Search Controllers & Services

TransactionController, AccountController, CustomerController & AuditController (/search) — MODIFIED

```
// Using GET + @ModelAttribute + @Valid combines URL query filters, Pageable, Sort, & Bean Validation!

// 1. TransactionController.java
@GetMapping("/search")
public ResponseEntity<PagedResponse<TransactionSummaryResponse>> searchTransactions(
```

```
        @Valid @ModelAttribute TransactionSearchRequest filter,
        @PageableDefault(size = 20, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable) {
    return ResponseEntity.ok(transactionService.searchTransactions(filter, pageable));
}

// 2. AccountController.java
@GetMapping("/search")
public ResponseEntity<PagedResponse<AccountSummaryResponse>> searchAccounts(
        @Valid @ModelAttribute AccountSearchRequest filter,
        @PageableDefault(size = 20, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable) {
    return ResponseEntity.ok(accountService.searchAccounts(filter, pageable));
}

// 3. CustomerController.java
@GetMapping("/search")
public ResponseEntity<PagedResponse<CustomerSummaryResponse>> searchCustomers(
        @Valid @ModelAttribute CustomerSearchRequest filter,
        @PageableDefault(size = 20, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable) {
    return ResponseEntity.ok(customerService.searchCustomers(filter, pageable));
}

// Inside ServiceImpl (e.g., TransactionServiceImpl using TransactionSpecification):
Pageable validated = SortValidator.validate(pageable, ALLOWED_SORT_FIELDS, DEFAULT_SORT);
return PagedResponse.from(repository.findAll(TransactionSpecification.withFilters(filter),
validated).map(mapper::toSummaryResponse));
```

> **Why `root.get("sourceAccount").get("id")` is Fast & Join-Safe:** Accessing the FK attribute via `root.get("customer").get("id")` references the foreign-key column `customer_id` directly in the `accounts` table *without* issuing an unnecessary SQL `INNER JOIN customers`! That prevents duplicate rows in `COUNT(*)` pagination queries and leverages Day 70's B-Tree indexes directly.

## 6. Verification & Testing Commands (curl)

| Search Scenario | HTTP Request | Expected Result |
|---|---|---|
| Filter Accounts by Type & Balance Range | `GET /api/v1/accounts/search?accountType=SAVINGS&minBalance=1000&sort=balance,desc` | `200 OK`, sorted savings accounts ≥ 1000. |
| Search Transactions by Date Window | `GET /api/v1/transactions/search?accountId=1&fromDate=2026-11-01T00:00:00&transactionType=DEPOSIT` | `200 OK`, filtered & paginated matches. |
| Invalid Range Cross-Check | `GET /api/v1/accounts/search?minBalance=5000&maxBalance=100` | `400 Bad Request` (Day 76 + Day 79 guard). |
| Wildcard Escape Protection | `GET /api/v1/customers/search?firstName=%25` | Escaped literal `\%` match — zero full-table dump! |